

Servizio di Messaggistica Client-Server

Introduzione: Architettura e Scelte Progettuali Chiave

Questo documento costituisce la specifica tecnica completa e l'analisi di progettazione per un'applicazione di **messaggistica** basata su un'architettura **Client-Server**. Sviluppato interamente in linguaggio C per sistemi operativi POSIX, il progetto si fonda su un insieme di scelte architetturali mirate a garantire robustezza, efficienza e isolamento del servizio. L'analisi che segue non si limiterà a descrivere il funzionamento del sistema, ma esplorerà il "perché" dietro ogni decisione implementativa. Il progetto "poggia" su tre pilastri logici fondamentali:

1. Gestione della Concorrenza tramite Modello "Thread-per-Client"

Per servire un numero potenzialmente elevato di client simultaneamente è stato adottato un modello dinamico in cui ogni nuova connessione accettata genera un thread dedicato. Questa scelta, pur avendo un leggero overhead nella creazione dei thread, garantisce la massima isolamento: un'operazione bloccante o un errore di un client non impatta sulla reattività degli altri. L'uso di `pthread_detach` assicura che le risorse del thread vengano rilasciate automaticamente alla disconnessione, prevenendo *memory leak* o *thread zombie*.

2. Protocollo di Comunicazione Testuale Line-Based con Tokenizzazione

La comunicazione tra client e server è governata da un protocollo semplice e ispezionabile, basato su righe di testo terminate da `\n` e campi delimitati dal carattere *pipe* (`|`). Per superare la natura di "stream" di TCP, è stata implementata la funzione `recv_line`. Questa funzione garantisce la ricezione atomica e sicura di ogni comando a livello applicativo, assicurando che l'intera riga logica venga processata correttamente prima di procedere alla tokenizzazione tramite `strtok`.

3. Persistenza della memoria con Strategia Ibrida (Write-Through + Write-Back)

Il sistema adotta una doppia strategia per ottimizzare sicurezza e performance. I dati sono gestiti in modo differenziato:

- **Dati Utente** (`users.txt`): questi dati critici sono gestiti con la strategia *Write-Through* e protetti da `file_mutex`. Ogni operazione di registrazione utente si riflette immediatamente sul disco per garantire la massima durabilità degli account e la coerenza persistente delle informazioni di autenticazione.
- **Dati Messaggi (Cache in Memoria)**: per eliminare il collo di bottiglia del disco e garantire alta reattività, l'intero archivio messaggi è mantenuto in un array dinamico in RAM (`global_cache`). La coerenza e l'accesso concorrente alla cache sono gestiti da un *Mutex leggero* (`cache_mutex`), che blocca le operazioni solo per la durata brevissima necessaria alla manipolazione della memoria. La persistenza su file (`messages.txt`) avviene solo in fase di *Save-on-Shutdown*, garantito tramite la registrazione di una funzione di *cleanup* (`atexit`).

Queste scelte implementative consentono di sviluppare un'interfaccia Client-Server che garantisce elevate performance per le operazioni di I/O frequenti (messaggi) e una *user-experience* accettabile e conforme con la richiesta progettuale.

Specifiche Funzionali Chiave

1. Gestione Utenti

Il sistema permette a nuovi utenti di registrarsi con un nome utente e una password (persistenza immediata su disco). Gli utenti esistenti possono accedere al sistema fornendo le proprie credenziali.

2. Concorrenza

Il server è in grado di gestire le richieste di più client contemporaneamente. L'accesso alle risorse condivise è gestito con due livelli di mutua esclusione:

- `cache_mutex`: Protegge l'accesso alla memoria RAM (archivio messaggi) garantendo operazioni velocissime.
- `file_mutex`: Protegge l'accesso al disco (solo per la registrazione utenti e le operazioni di load/save della cache), prevenendo *race condition* sul file system.

3. Interazione con il Servizio

- **Invio:** Le nuove operazioni di invio messaggio vengono gestite interamente in RAM (`global_cache`), garantendo una risposta quasi istantanea al client.
- **Lettura:** La visualizzazione della propria casella di posta avviene tramite scansione diretta della cache in memoria, assicurando una velocità di recupero dati massima (non è necessario accedere al disco).
- **Cancellazione:** La cancellazione dei messaggi è implementata come compattazione logica della struttura dati in RAM.

4. Robustezza

Il sistema implementa meccanismi per gestire la disconnessione anomala dei client. Per quanto riguarda il server, è garantita una chiusura pulita (*Graceful Shutdown*) in risposta a segnali esterni (SIGINT), durante la quale l'intero stato in RAM viene salvato su disco tramite la funzione registrata con `atexit()`.

MANUALE D'USO

Compilazione (gcc compiler)

Per compilare il progetto (versione POSIX) basta eseguire sulla riga di comando del terminale

```
make posix
```

Mentre per eseguire la pulizia degli eseguibili basta scrivere

```
make clean
```

Esecuzione

1. Avviare il Server

Per avviare il server, eseguire il seguente comando nel terminale:

```
./server
```

Il server si metterà in ascolto sulla porta `8080` (come definito nel file `server.c`). I file `users.txt` e `messages.txt`, necessari per la persistenza dei dati, verranno creati e gestiti direttamente nella directory corrente di esecuzione del server. Il terminale mostrerà il messaggio di conferma: `[Server] Avviato`

su porta 8080.

2. Avviare il Client

Per connettersi al server, aprire un nuovo terminale e avviare il client. Se il server è in esecuzione sulla stessa macchina usare il comando:

```
./client
```

3. Interagire con l'applicazione

Una volta stabilita la connessione, il client mostrerà un **menu testuale iniziale** per consentire l'autenticazione. L'utente potrà scegliere tra **Registrazione (2)**, **Accesso (1)** o **Uscita (3)**.

Dopo aver effettuato l'accesso con successo, sarà disponibile il **menu principale** per interagire con il servizio di messaggistica:

- **Leggi messaggi (2):** Richiede al server di inviare tutti i messaggi destinati all'utente loggato.
- **Cancella messaggi (3):** Rimuove tutti i messaggi precedentemente ricevuti e conservati per l'utente loggato.
- **Esci (4):** Invia il comando di disconnessione al server e chiude l'applicazione client.

4. Chiudere il Server

Per chiudere il server in modo pulito (*Graceful Shutdown*), premere **Ctrl+C** nel terminale in cui è in esecuzione.

Il server intercetterà il segnale **SIGINT** (*Signal Interrupt*) e avvierà una procedura di terminazione:

- Il server esegue la funzione `handle_sigint` che chiude il socket di ascolto (`server_fd`) e innesca la chiusura del programma (`exit(0)`).
- L'uscita del programma attiva automaticamente la funzione `server_shutdown_cleanup` registrata tramite `atexit()`.
- Questa funzione esegue l'operazione critica di `save_messages_to_file`, che riscrive l'intero contenuto della cache in RAM (`global_cache`) sul file `messages.txt`, garantendo che nessun messaggio sia perso.
- Questa chiusura pulita garantisce che la porta **8080** venga immediatamente liberata per un eventuale riavvio e che i thread client attivi vengano chiusi automaticamente dal sistema operativo.

GESTIONE DELLO STATO E DELLA DURABILITA' DEI DATI

La permanenza dei dati è una caratteristica fondamentale per un sistema di messaggistica, in cui le informazioni relative agli utenti e ai messaggi devono sopravvivere ai riavvii del server. Il progetto soddisfa tale requisito attraverso un meccanismo di persistenza basato su **file di testo**, gestito direttamente dal server tramite le funzioni di gestione utenti e messaggi.

Scelte di Progetto

La scelta di utilizzare file di testo semplici e leggibili è stata dettata dalla necessità di garantire semplicità, trasparenza e portabilità, evitando dipendenze esterne o meccanismi di persistenza complessi.

- **File degli Utenti (`users.txt`):** Questo file memorizza le credenziali degli utenti. Ogni riga

contiene una coppia `username:password`.

La gestione di questo file avviene tramite le funzioni `authenticate`, `user_exists` e `create_user`, ed è protetta da un *Mutex* dedicato (`file_mutex`) per prevenire *race condition* durante l'accesso concorrente.

- **File dei Messaggi** (`messages.txt`): Questo file rappresenta una copia persistente dello stato della cache dei messaggi.

Ogni riga descrive un singolo messaggio nel formato testuale:

```
recipient|sender|subject|body
```

Meccanismi di Caricamento e Salvataggio

1. Caricamento all'Avvio (Loading)

All'avvio del server, prima di iniziare ad accettare connessioni client, viene ripristinato lo stato dei messaggi precedentemente salvati.

Nel `main` del server viene invocata la funzione `load_messages_from_file()` che si occupa di:

- Aprire il file `messages.txt` in modalità lettura.
- Eseguire il parsing del file riga per riga.
- Ricostruire ogni messaggio all'interno di una struttura dati residente in RAM (`global_cache`), implementata come array dinamico di strutture `Message`.
- Allocare dinamicamente memoria aggiuntiva quando necessario tramite `realloc`.

Al termine di questa fase, l'intero archivio dei messaggi risulta disponibile in memoria centrale, pronto per essere utilizzato dal servizio senza ulteriori accessi al disco.

2. Salvataggio alla Chiusura (Saving)

Per motivi di performance, il salvataggio dei messaggi **non avviene a ogni modifica**, ma è demandato alla fase di spegnimento controllato del server. Questo comportamento implementa una strategia di persistenza **write-back**, consapevolmente scelta per ridurre il costo delle operazioni di I/O.

Il salvataggio è garantito da due meccanismi complementari:

- **Handler di Segnale** (`handle_sigint`): Il server installa un gestore per il segnale `SIGINT`. Alla ricezione del segnale (ad esempio tramite `Ctrl+C`), il server interrompe l'accettazione di nuove connessioni e avvia una procedura di terminazione controllata.
- **Funzione di Cleanup** (`atexit`): Nel `main` del server, la funzione `cleanup` viene registrata tramite `atexit(server_shutdown_cleanup)`. Questa funzione viene automaticamente eseguita alla terminazione del processo e si occupa di salvare l'intero contenuto della cache dei messaggi su disco tramite `save_messages_to_file`, di rilasciare correttamente la memoria allocata dinamicamente. La funzione `save_messages_to_file` itera sull'array `global_cache` e sovrascrive completamente il file `messages.txt` con lo stato aggiornato dei messaggi, garantendo la coerenza della persistenza.

STANDARDIZZAZIONE DELLO SCAMBIO DATI CLIENT-SERVER

In un'applicazione Client-Server è fondamentale che entrambe le parti concordino sulle dimensioni massime dei dati che possono essere scambiati (ad esempio lunghezza di `username`, `password` e messaggi) e sulle regole che definiscono il protocollo di comunicazione. Una discrepanza in tali

definizioni può portare a bug gravi, come buffer overflow, troncamento dei dati o errori di interpretazione dei comandi applicativi.

Centralizzazione delle Definizioni

Per affrontare questo problema, il progetto adotta una pratica di sviluppo solida: centralizzare tutte le definizioni comuni in file header condivisi, inclusi sia nel client che nel server.

1. COMMON/common.h:

Il file `common.h`, collocato in una directory condivisa esterna ai moduli client e server, contiene le definizioni comuni utilizzate da entrambe le componenti del sistema.

In particolare, in `common.h` sono definite le dimensioni massime dei dati scambiati tramite il protocollo testuale:

```
#define MAX_USERNAME_LEN    50
#define MAX_PASSWORD_LEN    50
#define MAX_SUBJECT_LEN     100
#define MAX_BODY_LEN        1024
#define MAX_MSG_LEN         2048
```

Lato Client

Queste costanti vengono utilizzate dal client per dichiarare i buffer che contengono l'input dell'utente. Ad esempio:

```
char user[MAX_USERNAME_LEN];
char pass[MAX_PASSWORD_LEN];
char subj[MAX_SUBJECT_LEN];
char body[MAX_BODY_LEN];
char buffer[MAX_MSG_LEN];
```

Lato Server

Sul server, le stesse costanti sono utilizzate per:

- definire la dimensione dei campi all'interno delle strutture dati (ad esempio nella struttura `Message`);
- dimensionare i buffer temporanei impiegati per la ricezione delle richieste dei client.

L'uso di un header condiviso assicura che client e server abbiano una visione identica dei limiti dimensionali, eliminando incoerenze e vulnerabilità legate alla gestione della memoria.

2. Protocollo di Comunicazione:

Il sistema adotta un protocollo testuale *line-based*, in cui ogni comando è rappresentato da una riga terminata dal carattere `\n` e i campi sono separati dal delimitatore `|`.

Esempi di comandi sono:

```
LOGIN|username|password
SEND|recipient|subject|body
READ
DELETE
QUIT
```

Per gestire correttamente la natura *stream-oriented* di TCP, sia il client che il server utilizzano una

funzione dedicata (`recv_line`) che garantisce la ricezione completa di una riga logica prima di procedere al *parsing* dei campi tramite *tokenizzazione*. Questo approccio evita ambiguità nella delimitazione dei messaggi e garantisce una corretta interpretazione dei comandi.

Considerazioni Progettuali

Sebbene un protocollo binario con header strutturati possa offrire maggiore efficienza e flessibilità, il protocollo testuale adottato rappresenta una scelta consapevole orientata a:

- semplicità di implementazione;
- facilità di debugging;
- maggiore trasparenza durante lo sviluppo e il testing.

La standardizzazione delle dimensioni dei dati tramite `common.h`, unita a un protocollo testuale ben definito, consente di ottenere un sistema robusto, sicuro e manutenibile, in cui client e server condividono un contratto di comunicazione coerente e globale.

AUTENTICAZIONE/ISCRIZIONE AL SISTEMA

Un sistema multiutente richiede un meccanismo affidabile per la gestione dell'identità degli utenti. Il progetto implementa le due operazioni fondamentali di **registrazione (iscrizione)** e **autenticazione (login)**, basate su un protocollo di comunicazione testuale client-server. La logica applicativa risiede nel server, mentre il client si occupa della raccolta delle credenziali e dell'interazione con l'utente.

Le funzionalità di autenticazione sono realizzate tramite:

- **Server:** funzioni `create_user`, `authenticate` e `user_exists`, protette da `file_mutex`.
- **Client:** gestione del flusso di login/registrazione nel menu iniziale.

Le credenziali sono persistite nel file `users.txt`.

Flusso della Registrazione

1. Client

L'utente seleziona l'opzione Registrazione dal menu iniziale. Il client acquisisce username e password tramite input standard, applicando una sanitizzazione di base (rimozione di newline e caratteri pipe).

2. Costruzione del Messaggio

Le credenziali vengono codificate in una singola riga di testo secondo il protocollo testuale:

```
REGISTER|username|password\n.
```

3. Invio al Sever

Il client invia la stringa al server tramite `send()` su socket TCP.

4. Server – Parsing del Comando

Il thread dedicato al client (`handle_client`) riceve il messaggio tramite `recv_line`, effettua la tokenizzazione con `strtok` e riconosce il comando `REGISTER`.

5. Logica di Registrazione (`create_user`)

- Il server acquisisce il `mutex file_mutex` per garantire accesso esclusivo al file `users.txt`.
- Verifica se l'utente esiste già tramite `user_exists`.
- Se l'utente non esiste, appende la coppia `username:password` al file.
- Rilascia il mutex e restituisce l'esito dell'operazione.

6. Risposta al Client (tramite protocollo testuale)

- `OK_REG` → registrazione avvenuta con successo.
- `FAIL_EXISTS` → username già presente.

7. Client

Il client interpreta la risposta e informa l'utente dell'esito dell'operazione.

Flusso dell'Autenticazione (Login)

Il flusso del login è molto simile a quello della registrazione.

1. Client

L'utente seleziona l'opzione *Login* e inserisce username e password.

2. Invio al Server

Il client invia il comando:

- `LOGIN|username|password\n`

1. Server – Autenticazione (`authenticate`)

- **Server** Il server acquisisce `file_mutex`.
- Legge il file `users.txt` riga per riga.
- Confronta username e password con le coppie memorizzate.
- Rilascia il mutex e restituisce il risultato.

2. Gestione dello Stato di Sessione

In caso di autenticazione riuscita:

- Il thread `handle_client` memorizza lo username nella struttura `ClientHandler`.
- Lo stato di autenticazione rimane valido per tutta la durata della connessione.
- Le operazioni successive (invio, lettura, cancellazione messaggi) vengono autorizzate sulla base di tale stato.

3. Risposta al client

- `OK` → login riuscito.
- `FAIL` → credenziali non valide.

4. Client

Alla ricezione di `OK`, il client sblocca il menu principale e consente l'accesso alle funzionalità della bacheca.

SICUREZZA DEI DATI E DEL SISTEMA

La sicurezza rappresenta un aspetto rilevante in un sistema multiutente, in particolare per quanto riguarda la gestione delle credenziali di accesso. Il progetto implementa **misure di sicurezza di base**, coerenti con la natura didattica del sistema, e presta particolare attenzione alla protezione delle password e alla gestione sicura della memoria.

Hashing delle Password

Il sistema non memorizza mai le password in chiaro sul disco. Questo è un principio fondamentale della sicurezza. Invece, utilizza una funzione di hash.

Implementazione (`users.c`)

La funzione `hash_password` utilizza l'algoritmo **djb2**, un hash non crittografico semplice e computazionalmente efficiente. Durante l'autenticazione, la password inserita dall'utente viene hashata con la stessa funzione e confrontata con il valore memorizzato.

Analisi della Sicurezza

Punto di Forza

- L'uso di un hash è meglio che memorizzare password in chiaro. Se il file `users.txt` venisse compromesso, un aggressore non otterrebbe direttamente le password degli utenti.
- La funzione di hash è deterministica e leggera, garantendo buone prestazioni anche in presenza di accessi concorrenti.
- L'accesso al file delle credenziali è protetto tramite un mutex (`file_mutex`), prevenendo *race condition* e corruzione dei dati.

Punto di Debolezza Critico

- L'algoritmo djb2 **non è un hash crittografico sicuro**. È stato progettato per le hash table, non per la sicurezza. I suoi principali difetti in questo contesto sono:
 1. Velocità di calcolo (vulnerabili ad attacchi di tipo *brute-force*)
 2. Mancanza di Randomicità

Protezione da Buffer Overflow

Il progetto presta attenzione a prevenire buffer overflow utilizzando le costanti definite in `common.h` e funzioni di libreria sicure dove possibile. Questo riduce il rischio di vulnerabilità legate alla corruzione della memoria.

GESTIONE DEI SEGNALI

La gestione dei segnali è un aspetto fondamentale per la robustezza di applicazioni a lunga esecuzione come un server concorrente o un client interattivo. Il progetto implementa una gestione dei segnali **semplice ma efficace**, con obiettivi differenti lato server e lato client.

Gestione dei Segnali sul Server

L'obiettivo principale del server è garantire una **chiusura pulita** (*graceful shutdown*), evitando la perdita di dati e assicurando la persistenza dello stato in memoria prima della terminazione del processo.

Implementazione (`server.c`)

Il server gestisce esplicitamente il segnale SIGINT (generato, ad esempio, dalla combinazione Ctrl+C).

1. Registrazione dell'Handler

Nel main del server viene registrato un gestore di segnale tramite la funzione standard `signal(SIGINT, handle_sigint)`.

2. Logica dell'Handler (`handle_sigint`)

Quando il segnale SIGINT viene ricevuto:

- L'handler non esegue operazioni complesse.
- imposta una variabile globale atomica (`running = 0`).
- chiude il socket di ascolto (`server_fd`), sbloccando la `accept`.

Questo approccio rispetta le regole di sicurezza degli handler di segnale, limitandosi a operazioni *async-signal-safe*.

3. Terminazione Controllata del Server

Il ciclo principale del server controlla periodicamente la variabile `running`:

```
while (running) {
    accept(...)
}
```

Quando `running` viene impostata a 0:

- Il ciclo di accettazione termina.
- Il `main` raggiunge la fine.
- Il processo termina normalmente.

4. Persistenza tramite `atexit`

Prima di entrare nel loop principale, il server registra una funzione di cleanup `atexit(server_shutdown_cleanup)`.

Alla terminazione del programma, questa funzione:

- Salva l'intera cache di messaggi su disco (`messages.txt`).
- Rilascia la memoria allocata.

In questo modo, anche una terminazione causata da SIGINT garantisce la persistenza dei dati e una chiusura ordinata del servizio.

Questa strategia bilancia la necessità di chiudere il server con quella di non interrompere il servizio per i client connessi.

Gestione dei Segnali sul Client

Sul client l'obiettivo è migliorare l'esperienza utente, prevenendo la chiusura accidentale dell'applicazione durante l'interazione.

Implementazione (`client.c`)

Nel client viene ignorato il segnale SIGPIPE, che può verificarsi quando il server chiude la connessione

mentre il client tenta di scrivere sul socket:

```
○ signal(SIGPIPE, SIG_IGN);
```

Questa scelta evita la terminazione improvvisa del processo client e consente di gestire l'errore di comunicazione in modo controllato tramite i valori di ritorno di `send`.

Terminazione del Client

La chiusura del client avviene **esclusivamente in modo esplicito**, quando l'utente seleziona l'opzione "Esci" dal menu. In tal caso:

- Viene inviato il comando `QUIT` al server.
- Il socket viene chiuso correttamente.
- Il programma termina in modo ordinato.

GESTIONE DELLE DISCONNESSIONI

La gestione corretta delle disconnessioni dei client è fondamentale per garantire la stabilità e la robustezza di un server concorrente a lunga esecuzione. Il progetto implementa un meccanismo semplice ed efficace per rilevare la chiusura delle connessioni e rilasciare correttamente tutte le risorse associate.

Rilevamento della Disconnessione

Ogni client connesso al server è gestito da un thread dedicato che esegue la funzione `handle_client`. All'interno di questa funzione è presente un ciclo principale che attende e processa i comandi inviati dal client.

Il rilevamento della disconnessione avviene tramite la funzione `recv_line`, che incapsula la chiamata di sistema `recv`.

```
ssize_t bytes = recv_line(sock, buffer, sizeof(buffer));  
if (bytes <= 0) break;
```

La semantica del valore di ritorno è la seguente:

- **Valore 0:** il client ha chiuso la connessione in modo ordinato.
- **Valore negativo (-1):** si è verificato un errore di comunicazione, tipicamente associato a una disconnessione anomala (es. chiusura forzata del client).

In entrambi i casi, il ciclo principale di `handle_client` viene interrotto, segnalando la fine della comunicazione con il client.

Pulizia delle Risorse

Una volta terminato il ciclo di gestione, il thread procede alla liberazione delle risorse associate al client:

1. Chiusura del Socket

Il socket associato al client viene chiuso tramite `close(sock)`, rilasciando il file descriptor e le risorse di sistema collegate alla connessione di rete.

2. Deallocazione della Struttura Client

La struttura dinamicamente allocata per memorizzare le informazioni del client (`ClientHandler`) viene liberata tramite `free`, evitando *memory leak*.

3. Terminazione del Thread

I thread client sono creati in modalità `detached` (`pthread_detach`), quindi la loro terminazione comporta automaticamente il rilascio delle risorse del thread senza la necessità di una `pthread_join`.

INTERFACCIA APPLICATIVA CLIENT-SERVER

Le funzionalità principali del servizio di messaggistica (visualizzazione, invio e cancellazione dei messaggi) sono implementate tramite un protocollo di comunicazione **testuale request-response**, semplice ma efficace, basato su comandi delimitati da caratteri pipe (`|`) e terminati da newline (`\n`). Il server elabora ogni richiesta all'interno della funzione `handle_client`, eseguita in un thread dedicato per ciascun client.

Visualizzazione della Bacheca (READ)

La visualizzazione dei messaggi consente a un utente autenticato di leggere tutti i messaggi a lui destinati, memorizzati nella cache in memoria del server.

1. Client

- L'utente seleziona l'opzione "Leggi messaggi". Il client invia al server il comando testuale:

```
READ\n
```

2. Server (`read_messages`)

- Il server acquisisce il mutex `cache_mutex` per garantire una lettura consistente della struttura dati condivisa.
- Scansiona l'intero array dinamico `global_cache`, selezionando solo i messaggi il cui campo `recipient` coincide con l'utente autenticato.
- Per ogni messaggio trovato, invia al client una riga nel formato:

```
FROM|mittente|oggetto|corpo\n
```

- Se non sono presenti messaggi, il server invia una stringa informativa:

```
Nessun nuovo messaggio.\n
```

- Al termine dell'invio, il server invia sempre un marcatore di fine:

```
END_READ\n
```

3. Client

- Il client entra in un ciclo di ricezione basato su `recv_line`.
- Stampa ogni riga ricevuta fino a quando incontra il token `END_READ`, che segnala la conclusione della risposta del server.

Questo meccanismo di invio sequenziale evita l'uso di buffer di grandi dimensioni e consente di gestire un numero arbitrario di messaggi senza spreco di memoria.

Invio di un Messaggio (SEND)

L'invio di un messaggio è progettato per essere un'operazione veloce, gestita interamente in memoria.

1. Client

- L'utente inserisce destinatario, oggetto e corpo del messaggio.
- I dati vengono assemblati in un'unica riga secondo il formato:

```
SEND|destinatario|oggetto|testo\n
```

2. Server (handle_client → save_message)

- Il server riceve il comando e ne effettua la tokenizzazione.
- Acquisisce il mutex `cache_mutex`.
- Inserisce il nuovo messaggio nell'array dinamico `global_cache`, memorizzando mittente, destinatario, oggetto e corpo.
- Rilascia il mutex.

3. Risposta

- Il server invia una conferma al client:

```
OK\
```

4. Client

- Riceve la risposta e notifica all'utente il successo dell'operazione.

Cancellazione dei Messaggi (DELETE)

La cancellazione consente a un utente di eliminare tutti i messaggi a lui destinati.

1. Client

- L'utente seleziona l'opzione di cancellazione.
- Il client invia il comando:

```
DELETE\n
```

2. Server (delete_messages)

- Il server acquisisce il mutex `cache_mutex`.
- Scansiona l'array `global_cache` e rimuove logicamente tutti i messaggi il cui destinatario corrisponde all'utente autenticato.
- L'operazione è implementata come una compattazione dell'array, evitando riallocazioni inutili.
- Rilascia il mutex.

3. Risposta

- Il server risponde con:

```
OK\n
```

4. Client

Riceve la conferma e informa l'utente dell'avvenuta cancellazione

CANCELLAZIONE DI UN MESSAGGIO, SICUREZZA ED EFFICIENZA

L'operazione di cancellazione dei messaggi è progettata per garantire sia la sicurezza dell'accesso sia una gestione efficiente della memoria, mantenendo il codice semplice, robusto e thread-safe.

Il sistema supporta due modalità di cancellazione:

- DELETE → elimina tutti i messaggi destinati all'utente autenticato
- DELETE_ONE → elimina un singolo messaggio specificando il suo ID univoco

Entrambe le operazioni sono gestite interamente lato server.

Logica di Autorizzazione

Nel progetto, la cancellazione dei messaggi è implicitamente autorizzata sulla base dell'identità dell'utente autenticato.

Il server, durante la gestione della connessione, associa il socket all'utente autenticato

```
(client->username).
```

Ogni richiesta di cancellazione utilizza questo valore come parametro di controllo.

Nel caso della cancellazione totale (DELETE), la funzione `delete_messages` rimuove esclusivamente i messaggi destinati all'utente:

```
if (strcmp(user, m->recipient) != 0) {  
    // il messaggio viene mantenuto  
}
```

Nel caso della cancellazione di un singolo messaggio (DELETE_ONE), la funzione `delete_specific_message` verifica sia:

- La corrispondenza del destinatario
- La corrispondenza dell'ID del messaggio

```
if (strcmp(user_clean, recipient_clean) == 0 && m->id == target_id) {  
    deleted = 1;  
    continue; // elimina il messaggio  
}
```

In questo modo:

- Un utente non può cancellare messaggi destinati ad altri
- Un utente non può cancellare un messaggio se non conosce il suo ID
- Il client non ha alcun controllo sulle autorizzazioni
- Tutta la logica di sicurezza risiede esclusivamente sul server

Questa scelta rispetta un principio fondamentale dei sistemi client-server, il client non è mai considerato attendibile.

Identificazione Unica dei Messaggi

Ogni messaggio possiede un identificativo univoco (`int id`), assegnato dal server tramite una variabile globale incrementale:

```
int next_message_id = 1;
```

Quando un messaggio viene creato:

```
m->id = next_message_id++;
```

L'ID viene:

- Salvato nel file persistente (`messages.txt`)
- Ricaricato in fase di avvio
- Inviato al client durante l'operazione READ

Questo meccanismo garantisce:

- Unicità dei messaggi
- Cancellazione selettiva sicura
- Assenza di ambiguità tra messaggi con stesso oggetto o contenuto

Implementazione della Cancellazione e Analisi di Efficienza

I messaggi sono memorizzati in un array dinamico:

```
global_cache.array
```

La cancellazione non utilizza riallocazioni immediate della memoria, ma viene implementata tramite una compattazione logica dell'array.

Algoritmi utilizzati

1) *Scansione Lineare* — $O(N)$

L'intero array viene attraversato una sola volta.

Per ogni messaggio:

- Se non deve essere cancellato → viene mantenuto
- Se deve essere cancellato → viene scartato

2) *Compattazione In-Place* — $O(N)$

Viene utilizzato un indice di scrittura separato:

```
size_t write_idx = 0;
```

Durante la scansione:

```
if (write_idx != read_idx)
    global_cache.array[write_idx] = global_cache.array[read_idx];
    write_idx++;
```

In questo modo:

- I messaggi validi vengono riscritti nella stessa struttura dati
- Non vengono creati "buchi" nell'array
- Non viene effettuata alcuna nuova allocazione

3) Aggiornamento del Contatore

Al termine dell'operazione:

```
global_cache.count = write_idx;
```

Il numero effettivo di messaggi viene aggiornato coerentemente.

Thread Safety

Tutte le operazioni sulla cache sono protette tramite mutua esclusione:

```
EnterCriticalSection(&cache_mutex);  
...  
LeaveCriticalSection(&cache_mutex);
```

Questo garantisce:

- Assenza di race condition
- Coerenza dei dati
- Corretto funzionamento in presenza di più client concorrenti

Gestione dei Formati di Linea (Windows)

Poiché il server utilizza socket su ambiente Windows, i comandi possono contenere terminazioni di riga nel formato `\r\n`.

Per evitare errori di parsing o blocchi del client, le stringhe ricevute vengono ripulite prima dell'elaborazione:

```
id_str[strcspn(id_str, "\r\n")] = '\0';
```

Questo garantisce robustezza nel parsing dei comandi (DELETE_ONE) e corretto confronto delle stringhe.

Analisi di Efficienza

1) Complessità Temporale

L'operazione di cancellazione ha complessità $O(N)$, dove N è il numero totale di messaggi presenti in cache.

La scansione viene effettuata una sola volta, senza ricerche annidate.

2) Complessità Spaziale

L'operazione avviene interamente in-place:

- Non viene allocata memoria aggiuntiva
- Non vengono create strutture temporanee significative

L'array dinamico viene semplicemente compattato e la complessità spaziale è quindi $O(1)$.

GESTIONE STREAM TCP

TCP è un protocollo di trasporto orientato allo stream, affidabile e orientato alla connessione. L'affidabilità garantisce che i dati arrivino senza errori e nell'ordine corretto, ma non garantisce che i dati inviati con una singola chiamata a `send()` vengano ricevuti con una singola chiamata a `recv()`. Per questo motivo, un'applicazione deve implementare una logica a livello applicativo per delimitare correttamente i messaggi.

Protocollo Line-Based e Funzione `recv_line`

Il progetto affronta la natura “a flusso” di TCP adottando un protocollo testuale *line-based*, in cui ogni messaggio applicativo è terminato da un carattere newline (`\n`).

Per garantire la ricezione corretta di una richiesta completa, è stata implementata la funzione helper `recv_line`, utilizzata sia dal server sia dal client.

```
ssize_t recv_line(int sock, char *buffer, size_t size);
```

Funzionamento di `recv_line`

1. La funzione legge il flusso TCP un byte alla volta tramite chiamate successive a `recv`.
2. I byte ricevuti vengono accumulati in un buffer fino a quando:
 - Viene incontrato il carattere `\n`, che segna la fine del messaggio applicativo.
 - Oppure si verifica la chiusura della connessione o un errore.
3. Una volta rilevato `\n`, la funzione termina la stringa con `\0` e restituisce la lunghezza della riga ricevuta.

Questo meccanismo garantisce che:

- Ogni comando venga ricevuto per intero.
- Non si verifichino troncamenti o concatenazioni di messaggi.
- Il parsing tramite `strtok` sia sempre corretto.

Gestione dell'Invio dei Dati

L'invio dei dati avviene tramite chiamate a `send`, utilizzando messaggi testuali completi già formattati e terminati da `\n`.

Poiché le stringhe inviate hanno dimensioni contenute (comandi e messaggi testuali), una singola `send` è sufficiente nella pratica.

Eventuali scritture parziali sono accettabili, poiché il protocollo *line-based* consente comunque al destinatario di ricostruire correttamente il messaggio tramite `recv_line`.

Per evitare la terminazione del processo in caso di scrittura su socket chiuso, il client ignora il segnale `SIGPIPE`.

```
signal(SIGPIPE, SIG_IGN);
```

Robustezza della Comunicazione

L'uso combinato di:

- Protocollo testuale delimitato da newline.
- Funzione `recv_line`.
- Parsing controllato dei comandi.

Rende la comunicazione robusta e affidabile a livello applicativo, pur senza l'uso di funzioni come `send_all` o `recv_all`.

Questa scelta semplifica l'implementazione, riduce la complessità del codice e risulta adeguata per un sistema di messaggistica interattivo con payload di dimensioni moderate.

FUTURE WORKS

Il progetto, pur presentando una base solida e funzionalmente corretta che garantisce una *user-experience* adeguata, offre numerosi spunti di miglioramento e di estensione futura. Tali evoluzioni riguardano principalmente la gestione della persistenza dei dati, la scalabilità del sistema e il rafforzamento delle misure di sicurezza.

Evoluzione della Gestione dei Messaggi

Attualmente i messaggi sono mantenuti in un'unica struttura in memoria e persistiti su un singolo file di testo al momento dello shutdown del server. Una possibile estensione consiste in:

- **Suddivisione dei messaggi su più file**, organizzati in directory basate su intervalli temporali (ad esempio per giorno o per mese).
Questa soluzione consentirebbe di migliorare la scalabilità del sistema e di evitare che un singolo file di persistenza cresca eccessivamente nel tempo.
- **Filtraggio temporale dei messaggi**, permettendo al client di richiedere solo messaggi appartenenti a uno specifico intervallo temporale.
Ciò ridurrebbe i tempi di risposta e migliorerebbe l'esperienza utente in presenza di archivi di grandi dimensioni.
- **Caricamento parziale della cache in memoria**, evitando di caricare l'intero archivio dei messaggi all'avvio del server.
Meccanismi di *lazy loading* o *caching selettivo* consentirebbero di sfruttare meglio la località dei dati e ridurre il consumo di memoria.

Estensioni Architetture per Alta Concorrenza

Nel caso in cui l'applicazione dovesse evolvere per gestire un numero molto elevato di client concorrenti, sarebbe possibile introdurre ulteriori miglioramenti architetturali. In particolare, il modello attuale *thread-per-client* potrebbe essere sostituito o affiancato da una **thread pool**, consentendo di limitare e controllare il numero massimo di thread attivi e di ridurre l'overhead legato alla creazione e distruzione dinamica dei thread.

Inoltre, il protocollo di comunicazione testuale attualmente adottato potrebbe essere rimpiazzato da un **protocollo binario strutturato**, basato su header e payload a lunghezza prefissata. Questa soluzione permetterebbe una gestione più efficiente dello stream TCP, riducendo l'overhead di parsing delle stringhe e migliorando le prestazioni complessive in scenari ad alta intensità di traffico.

Tali scelte, sebbene più complesse da implementare, rappresenterebbero un'evoluzione naturale del progetto verso contesti applicativi più vicini a sistemi di produzione ad alta scalabilità.

Miglioramenti alla Sicurezza

Dal punto di vista della sicurezza, il progetto implementa alcune misure di base, ma esistono diversi margini di miglioramento:

- **Adozione di funzioni di hash crittograficamente sicure** (come `bcrypt`, `sCrypt` o `Argon2`) in sostituzione dell'algoritmo `djb2`, che non è progettato per la protezione delle credenziali

in contesti reali.

- **Cifratura della comunicazione client-server**, ad esempio tramite TLS, per proteggere i dati trasmessi da intercettazioni e attacchi *man-in-the-middle*.
- **Rafforzamento dei controlli di autorizzazione**, introducendo politiche più granulari e verifiche aggiuntive lato server.

CONCLUSIONI

In questa relazione è stato descritto un servizio di messaggistica client-server implementato in linguaggio C per sistemi POSIX, progettata per garantire correttezza funzionale, semplicità architetturale e una gestione robusta delle risorse.

Il progetto dimostra l'applicazione coerente di concetti fondamentali dei sistemi operativi e della programmazione di rete, tra cui:

- Gestione concorrente dei client tramite thread dedicati.
- Sincronizzazione dell'accesso alle risorse condivise mediante mutex.
- Persistenza dei dati su file
- Protocollo di comunicazione applicativo testuale *line-based*.
- Gestione corretta delle disconnessioni.
- Misure di sicurezza di base, come l'hashing delle password e la prevenzione dei buffer overflow.

Pur non essendo pronto per una distribuzione in un ambiente di produzione, il sistema rappresenta una **base solida e ben strutturata** per futuri sviluppi. Le scelte progettuali adottate privilegiano la chiarezza, la manutenibilità e la coerenza con gli obiettivi didattici del progetto, evitando complessità non necessarie.

Gli obiettivi prefissati sono stati raggiunti con successo, anche grazie all'introduzione di funzionalità sperimentali che hanno permesso di valutare la flessibilità e le potenzialità evolutive dell'architettura. Le possibilità di miglioramento individuate in termini di scalabilità, sicurezza e prestazioni rappresentano linee di sviluppo naturali per l'evoluzione futura del sistema e confermano la validità dell'approccio progettuale adottato.